

第五章：中断与异常

一、核心概念

1.1 中断 vs 异常 vs 陷阱

这三者都是"打断 CPU 正常执行流"的事件，但来源不同：

类型	来源	同步/异步	举例
中断 (Interrupt)	外部硬件设备	异步	键盘输入、磁盘完成、定时器到期
异常 (Exception)	CPU 执行指令时的错误	同步	缺页 (Page Fault)、除零、非法指令
陷阱 (Trap)	软件主动触发	同步	系统调用 (ecall/syscall 指令)

关键区别：中断是异步的（随时可能发生），异常和陷阱是同步的（由当前执行的指令触发）。

1.2 中断控制器

多个外设的中断线需要汇聚、排优先级、分发给 CPU 核。这是中断控制器的职责：

架构	中断控制器	特点
RISC-V	PLIC (Platform-Level Interrupt Controller)	外部中断路由
RISC-V	CLINT (Core-Local Interruptor)	定时器 + 核间中断
x86	Local APIC + I/O APIC	per-CPU + 全局
ARM	GIC (Generic Interrupt Controller)	GICv2/v3/v4

1.3 Trap 的完整生命周期

事件触发 → CPU 自动操作（保存 PC 到 sepc, 设 scause）

- 跳转到 stvec 指向的处理函数
- 保存上下文（寄存器等）
- 判断原因并处理
- 恢复上下文
- sret 返回（PC = sepc）

1.4 用户/内核态切换的难题

用户态和内核态使用不同的页表。trap 发生时需要切换页表，但切换的瞬间 PC 还在执行代码——如果代码页在新页表中没有映射，CPU 就会立即再次 fault。

解决方案：Trampoline 页——一个在所有页表中都映射到相同虚拟地址的跳板页。切换页表时 PC 在 trampoline 页上，无论切到哪个页表，代码都能继续执行。

1.5 Page Fault 的分类

Fault 类型	RISC-V scause	触发条件
Instruction Page Fault	12	取指令时页无效/无执行权限
Load Page Fault	13	读数据时页无效/无读权限
Store Page Fault	15	写数据时页无效/无写权限/COW

Page Fault 是惰性内存管理的核心驱动力——它将"分配/加载物理页"的工作推迟到实际访问时刻。

二、基本推论

- 推论 1：中断处理必须快。** 中断处理期间通常关闭了同级或低级中断，长时间处理会导致其他设备响应延迟。因此产生了"上半部/下半部"的分离思想。
- 推论 2：Trampoline 是页表切换的必然要求。** 只要用户态和内核态使用不同页表，就需要一段在两个页表中都有效的代码作为跳板。这是一个普遍的架构约束，不是某个 OS 的特殊设计。
- 推论 3：Page Fault handler 是虚拟内存系统最关键的函数。** 惰性映射、COW、mmap 文件、swap 换入——所有延迟操作都在这里被"兑现"。它的正确性直接决定系统的稳定性。
- 推论 4：异常处理中不能再触发不可处理的异常。** 如果内核态发生 Page Fault（内核地址都是静态映射，不应缺页），说明出了严重 bug，必须 panic。

三、Linux / 通用实现

3.1 Linux 的中断处理：上半部/下半部

Linux 将中断处理分为两个阶段：

- **上半部（Top Half / Hardirq）**：在中断上下文中执行，关中断，只做最紧急的工作（读设备寄存器、清中断标志、调度下半部）
- **下半部（Bottom Half）**：延后执行，允许中断重入

下半部机制	执行上下文	特点
softirq	软中断上下文	高优先级，per-CPU，不可睡眠
tasklet	softirq 基础上	单次执行保证，不可睡眠
workqueue	内核线程上下文	可睡眠，可调度
threaded irq	专用内核线程	可睡眠，现代推荐

3.2 Linux 的 Page Fault 处理

```
do_page_fault() → handle_mm_fault()
  → __handle_mm_fault()
    → handle_pte_fault()
      |— pte 不存在：
      |   |— VMA 有 vm_ops->fault → do_fault()（文件映射）
```

```

├── 匿名映射 → do_anonymous_page()
├── swap 条目 → do_swap_page()
└── pte 存在但写保护:
    └── do_wp_page() (COW)

```

3.3 x86 的中断机制

x86 使用 **IDT (Interrupt Descriptor Table)** 代替 RISC-V 的单一 stvec:

- IDT 有 256 个条目，每个条目指定一个中断/异常的处理函数
- 通过 **LIDT** 指令加载 IDT 基址
- 中断发生时 CPU 自动查 IDT，进行特权级切换（ring 3→ring 0），切换到内核栈

APIC 架构:

- **Local APIC**: 每个 CPU 核一个，处理定时器和 IPI
- **I/O APIC**: 连接外设中断线，路由到指定 CPU 的 Local APIC

四、RUOS 的实现

核心文件

- **src/trap/trampoline.S** — 用户/内核态跳板页 (uservec / userret)
- **src/trap/trap.c** — C 语言异常处理主逻辑
- **src/trap/kernelvec.S** — 内核态异常入口

4.1 Trampoline 页

映射到 **TRAMPOLINE = MAXVA - PGSIZE**，在所有页表中地址一致。

```

用户空间页表:          内核页表:
  TRAMPOLINE → 物理页X   TRAMPOLINE → 物理页X ← 同一物理页

```

4.2 uservec (用户→内核)

```

uservec:
    csrrw a0, sscratch, a0    # 交换 a0 和 sscratch
    # 现在 a0 = trapframe 地址, sscratch = 用户 a0

    # 保存所有 31 个用户寄存器到 trapframe
    sd ra, 40(a0)
    sd sp, 48(a0)
    # ... (所有寄存器)

    # 从 trapframe 加载内核上下文
    ld sp, 8(a0)               # kernel_sp
    ld tp, 32(a0)              # kernel_hartid

```

```
ld t0, 16(a0)      # kernel_trap (usertrap)
ld t1, 0(a0)       # kernel_satp

# 切换到内核页表
sfence.vma zero, zero
csrw satp, t1
sfence.vma zero, zero

jr t0              # 跳到 usertrap()
```

关键洞察：切换页表后代码还能继续执行，因为 trampoline 在两个页表中地址一样。

4.3 usertrap() 的三条路径

```
usertrap():
    cause = r_scause();

    if (cause == ECODE_SYSCALL) {
        p->trapframe->epc += 4; // ecall 是 4 字节指令
        syscall_handler();
    }
    else if (devintr() != 0) {
        // 外部中断或定时器中断
    }
    else {
        // Page Fault
        va = r_stval();
        if (STORE_PAGE_FAULT):
            零页分配 → COW → VMA → kill(SIGSEGV)
        if (LOAD_PAGE_FAULT):
            VMA 惰性加载 → kill
        if (INST_PAGE_FAULT):
            VMA 惰性加载 → kill
    }
    usertrapret();
```

Page Fault 优先级链：零页 → COW → VMA → 杀进程。这个顺序很重要——先检查轻量级情况，最后才检查重量级的文件映射。

4.4 定时器中断路径

```
sbi_set_timer(val) → CLINT → M 态中断 → OpenSBI
→ 设 sip.SSIP → mret 回 S 态
→ S 态感知 SSIP → kernelvec/usertrap → devintr()
→ yield() → 清 sip.SSIP
```

RUOS 不直接操作 CLINT (M 态)，通过 SBI ecall 设置定时器。

4.5 内核态 vs 用户态异常对比

	用户态	内核态
入口	trampoline→uservec	kernelvec
处理	usertrap()	kerneltrap()
栈	切到内核栈	继续用当前栈
页表	切到内核页表	不切换
Page Fault	尝试处理	panic
返回	usertrapret→userret	恢复寄存器→sret

对比 Linux：Linux 的内核态 Page Fault 不一定 panic——如果 fault 地址在 `fixup_exception` 表中（用于 `copy_from_user` 等函数），可以优雅恢复。RUOS 简化为直接 panic。

五、八股 / 面试高频问题

Q: 中断和异常的区别？ A: 中断来自外部设备（异步），异常是 CPU 执行指令时触发的（同步）。中断可以被屏蔽（关中断），异常不能。

Q: 什么是中断上下文？为什么不能睡眠？ A: 中断上下文没有对应的进程上下文（没有 `task_struct`），不能被调度。如果在中断处理中调用 `sleep()`，没有进程可以被唤醒继续处理。

Q: 为什么 Linux 要区分上半部和下半部？ A: 上半部关中断执行，必须快速完成，否则其他中断得不到响应。耗时操作放到下半部（`softirq/workqueue`），允许中断重入。

Q: Page Fault 有哪几种？分别怎么处理？ A: 三种：取指（instruction）、读（load）、写（store）。根据 VMA 信息决定：匿名页 → 分配零页；文件映射 → 从文件读取；COW → 复制页；非法访问 → SIGSEGV。

Q: TLB 是什么？什么时候需要刷新？ A: TLB（Translation Lookaside Buffer）是页表的硬件缓存。切换页表（进程切换）、修改页表项（`munmap/mprotect`）时需要 `sfence.vma` / `invlpg` 刷新 TLB。多核还需要 TLB shootdown（IPI 通知其他核刷新）。

六、项目贡献亮点

- 实现了完整的 Page Fault 分层处理链（零页→COW→VMA），覆盖了所有惰性映射场景
- trampoline 机制实现了 S/U 态的安全切换，理解页表切换时的代码连续性问题
- 通过 SBI ecall 间接处理定时器中断，理解 M/S 态中断委托机制

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
trampoline	固定虚拟地址共享页	类似（VDSO/trampoline）	xv6 标准方案
Page Fault	分层：零页→COW→VMA	handle_mm_fault（更复杂）	覆盖所有惰性场景

决策	RUOS 选择	Linux 做法	权衡
内核缺页	panic	fixup_exception 恢复	教学 OS 合理简化
中断处理	全在中断上下文	上半部/下半部分离	简单但可能延迟较高
定时器	SBI ecall	直接操作 CLINT/APIC	避免 M 态代码